

being able to create strategy indicators for trading.

11.2.5 Next Steps

In the next section we are going to take a look at the Generalised Autoregressive Conditional Heteroscedasticity (GARCH) model and use it to explain more of the serial correlation in certain equities and equity index series.

Once we have discussed GARCH we will be in a position to combine it with the ARIMA model and create signal indicators and thus a basic quantitative trading strategy.

11.3 Volatility

The main motivation for studying conditional heteroskedasticity in finance is that of **volatility of asset returns**. Volatility is an incredibly important concept in finance because it is highly synonymous with *risk*.

Volatility has a wide range of applications in finance:

- **Options Pricing** - The Black-Scholes model for options prices is dependent upon the volatility of the underlying instrument
- **Risk Management** - Volatility plays a role in calculating the VaR of a portfolio, the Sharpe Ratio for a trading strategy and in determination of leverage
- **Tradeable Securities** - Volatility can now be traded directly by the introduction of the CBOE Volatility Index (VIX), and subsequent futures contracts and ETFs

If we can effectively forecast volatility then we will be able to price options more accurately, create more sophisticated risk management tools for our algorithmic trading portfolios and even design new systematic strategies that trade volatility directly.

We are now going to turn our attention to conditional heteroskedasticity and discuss what it means.

11.4 Conditional Heteroskedasticity

Let us first discuss the concept of **heteroskedasticity** and then examine the "conditional" part.

If we have a collection of random variables, such as elements in a time series model, we say that the collection is *heteroskedastic* if there are certain groups, or subsets, of variables within the larger set that have a different *variance* from the remaining variables.

For instance, in a non-stationary time series that exhibits seasonality or trending effects, we may find that the variance of the series increases with the seasonality or the trend. This form of *regular* variability is known as *heteroskedasticity*.

However, in finance there are many reasons why an increase in variance is correlated to a further increase in variance.

For instance, consider the prevalence of downside portfolio protection insurance employed by long-only fund managers. If the equities markets were to have a particularly challenging day (i.e. a substantial drop!) it could trigger automated risk management sell orders, which

would further depress the price of equities within these portfolios. Since the larger portfolios are generally highly correlated anyway, this could trigger significant downward volatility.

These "sell-off" periods, as well as many other forms of volatility that occur in finance, lead to heteroskedasticity that is *serially correlated* and hence *conditional* on periods of increased variance. Thus we say that such series are **conditional heteroskedastic**.

One of the challenging aspects of conditional heteroskedastic series is that if we were to plot the correlogram of a series with volatility we might still see what appears to be a realisation of *stationary* discrete white noise. That is, the volatility itself is hard to detect purely from the correlogram. This is despite the fact that the series is most definitely *non-stationary* as its variance is not constant in time.

We are going to describe a mechanism for detecting conditional heteroskedastic series in this chapter and then use the ARCH and GARCH models to account for it, ultimately leading to more realistic forecasting performance, and thus more profitable trading strategies.

11.5 Autoregressive Conditional Heteroskedastic Models

We have now discussed conditional heteroskedasticity (CH) and its importance within financial series. We want a class of models that can incorporate CH in a natural way. We know that the ARIMA model does not account for CH, so how can we proceed?

Well, how about a model that utilises an *autoregressive process for the variance itself*? That is, a model that actually accounts for the changes in the variance over time using past values of the variance.

This is the basis of the Autoregressive Conditional Heteroskedastic (ARCH) model. We will begin with the simplest possible case, namely an ARCH model that depends solely on the previous variance value in the series.

11.5.1 ARCH Definition

Definition 11.5.1. Autoregressive Conditional Heteroskedastic Model of Order Unity.

A time series $\{\epsilon_t\}$ is given at each instance by:

$$\epsilon_t = \sigma_t w_t \quad (11.5)$$

Where $\{w_t\}$ is discrete white noise, with zero mean and unit variance, and σ_t^2 is given by:

$$\sigma_t^2 = \alpha_0 + \alpha_1 \epsilon_{t-1}^2 \quad (11.6)$$

Where α_0 and α_1 are parameters of the model.

We say that $\{\epsilon_t\}$ is an *autoregressive conditional heteroskedastic model of order unity*, denoted by ARCH(1). Substituting for σ_t^2 , we receive:

$$\epsilon_t = w_t \sqrt{\alpha_0 + \alpha_1 \epsilon_{t-1}^2} \quad (11.7)$$

11.5.2 Why Does This Model Volatility?

I personally find the above "formal" definition lacking in motivation as to how it introduces volatility. However, you can see how it is introduced by squaring both sides of the previous equation:

$$\text{Var}(\epsilon_t) = E[\epsilon_t^2] - (E[\epsilon_t])^2 \quad (11.8)$$

$$= E[\epsilon_t^2] \quad (11.9)$$

$$= E[w_t^2] E[\alpha_0 + \alpha_1 \epsilon_{t-1}^2] \quad (11.10)$$

$$= E[\alpha_0 + \alpha_1 \epsilon_{t-1}^2] \quad (11.11)$$

$$= \alpha_0 + \alpha_1 \text{Var}(\epsilon_{t-1}) \quad (11.12)$$

Where I have used the definitions of the variance $\text{Var}(x) = E[x^2] - (E[x])^2$ and the linearity of the expectation operator E , along with the fact that $\{w_t\}$ has zero mean and unit variance.

Thus we can see that the variance of the series is simply a linear combination of the variance of the prior element of the series. Simply put, *the variance of an ARCH(1) process follows an AR(1) process.*

It is interesting to compare the ARCH(1) model with an AR(1) model. Recall that the latter is given by:

$$x_t = \alpha_0 + \alpha_1 x_{t-1} + w_t \quad (11.13)$$

You can see that the models are similar in form with the exception of the white noise term.

11.5.3 When Is It Appropriate To Apply ARCH(1)?

What approach can be taken in order to determine whether an ARCH(1) model is appropriate to apply to a series?

Consider that when we were attempting to fit an AR(1) model we were concerned with the decay of the first lag on a correlogram of the series. However if we apply the same logic to the *square* of the residuals and see whether we can apply an AR(1) to these squared residuals then we have an indication that an ARCH(1) process may be appropriate.

Note that ARCH(1) should only ever be applied to a series that has already had an appropriate model fitted sufficient to leave the residuals looking like discrete white noise. Since we can only tell whether ARCH is appropriate or not by *squaring* the residuals and examining the correlogram, we also need to ensure that the mean of the residuals is zero.

Crucially ARCH should only ever be *applied to series that do not have any trends or seasonal effects*, which are those that have no evident serial correlation. ARIMA is often applied to such a series (or even Seasonal ARIMA) at which point ARCH may be a good fit.

11.5.4 ARCH(p) Models

It is straightforward to extend ARCH to higher order lags. An ARCH(p) process is given by:

$$\epsilon_t = w_t \sqrt{\alpha_0 + \sum_{i=1}^p \alpha_i \epsilon_{t-i}^2} \quad (11.14)$$

You can think of ARCH(p) as applying an AR(p) model to the variance of the series.

An obvious question to ask at this stage is if we are going to apply an AR(p) process to the variance, why not a Moving Average MA(q) model as well? Or a mixed model such as ARMA(p,q)?

This is actually the motivation for the *Generalised* ARCH model, known as GARCH, which we will now define and discuss.

11.6 Generalised Autoregressive Conditional Heteroskedastic Models

11.6.1 GARCH Definition

Definition 11.6.1. Generalised Autoregressive Conditional Heteroskedastic Model of Order p, q.

A time series $\{\epsilon_t\}$ is given at each instance by:

$$\epsilon_t = \sigma_t w_t \quad (11.15)$$

Where $\{w_t\}$ is discrete white noise, with zero mean and unit variance, and σ_t^2 is given by:

$$\sigma_t^2 = \alpha_0 + \sum_{i=1}^q \alpha_i \epsilon_{t-i}^2 + \sum_{j=1}^p \beta_j \sigma_{t-j}^2 \quad (11.16)$$

Where α_i and β_j are parameters of the model.

We say that $\{\epsilon_t\}$ is a *generalised autoregressive conditional heteroskedastic model of order p,q*, denoted by GARCH(p,q).

Hence this definition is similar to that of ARCH(p) with the exception that we are adding moving average terms. That is, the value of σ^2 at t , σ_t^2 , is dependent upon previous σ_{t-j}^2 values.

Thus GARCH is the "ARMA equivalent" of ARCH, which only has an autoregressive component.

11.6.2 Simulations, Correlograms and Model Fittings

We are going to begin with the simplest possible case of the model—GARCH(1,1). This means we are going to consider a single autoregressive lag and a single "moving average" lag. The model is given by the following:

$$\epsilon_t = \sigma_t w_t \quad (11.17)$$

$$\sigma_t^2 = \alpha_0 + \alpha_1 \epsilon_{t-1}^2 + \beta_1 \sigma_{t-1}^2 \quad (11.18)$$

Note that it is necessary for $\alpha_1 + \beta_1 < 1$ otherwise the series will become unstable.

We can see that the model has three parameters, namely α_0 , α_1 and β_1 . Let us set $\alpha_0 = 0.2$, $\alpha_1 = 0.5$ and $\beta_1 = 0.3$.

To create the GARCH(1,1) model in R we need to perform a similar procedure as for the original random walk simulations. We need to create a vector **w** to store our random white noise values, a separate vector **eps** to store our time series values and finally a vector **sigsq** to store the ARMA variances.

We can use the R **rep** command to create a vector of zeros that we will populate with our GARCH values:

```
> set.seed(2)
> a0 <- 0.2
> a1 <- 0.5
> b1 <- 0.3
> w <- rnorm(10000)
> eps <- rep(0, 10000)
> sigsq <- rep(0, 10000)
> for (i in 2:10000) {
>   sigsq[i] <- a0 + a1 * (eps[i-1]^2) + b1 * sigsq[i-1]
>   eps[i] <- w[i]*sqrt(sigsq[i])
> }
```

At this stage we have generated our GARCH model using the aforementioned parameters over 10,000 samples. We are now in a position to plot the correlogram, which is given in Figure [11.7](#).

```
> acf(eps)
```

Notice that the series looks like a realisation of a discrete white noise process.

However, if we plot correlogram of the square of the series, as given in Figure [11.8](#)

```
> acf(eps^2)
```

we see substantial evidence of a conditionally heteroskedastic process via the decay of successive lags.

As in the previous chapter we now want to try and fit a GARCH model to this simulated series to see if we can recover the parameters. Thankfully a helpful library called **tseries** provides the **garch** command to carry this procedure out:

```
> require(tseries)
```

We can then use the **confint** command to produce confidence intervals at the 97.5% level for the parameters:

```
> eps.garch <- garch(eps, trace=FALSE)
> confint(eps.garch)
```

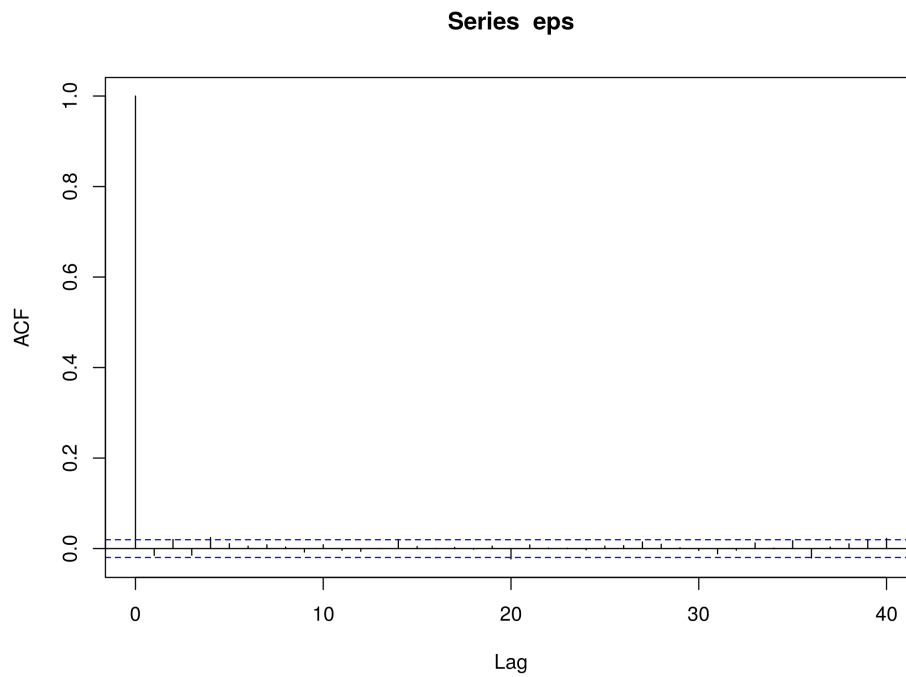


Figure 11.7: Correlogram of a simulated GARCH(1,1) model with $\alpha_0 = 0.2$, $\alpha_1 = 0.5$ and $\beta_1 = 0.3$

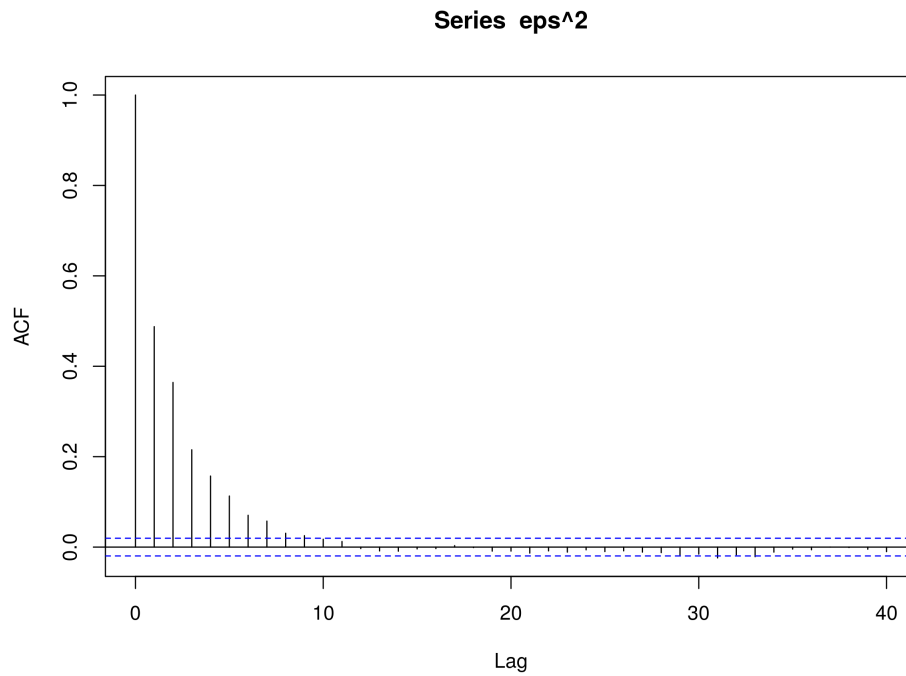


Figure 11.8: Correlogram of a simulated GARCH(1,1) models squared values with $\alpha_0 = 0.2$, $\alpha_1 = 0.5$ and $\beta_1 = 0.3$

```

      2.5 %    97.5 %
a0 0.1786255 0.2172683
a1 0.4271900 0.5044903
b1 0.2861566 0.3602687

```

We can see that the true parameters all fall within the respective confidence intervals.

11.6.3 Financial Data

Now that we know how to simulate and fit a GARCH model we are going to apply the procedure to some financial series. In particular let us try fitting ARIMA and GARCH to the FTSE100 index of the largest UK companies by market capitalisation. Yahoo Finance uses the symbol `^FTSE` for the index. We can use `quantmod` to obtain the data:

```

> require(quantmod)
> getSymbols("^FTSE")

```

We can then calculate the differences of the log returns of the closing price:

```

> ftrt = diff(log(Cl(FTSE)))

```

Let us plot the values as given in Figure 11.9.

```

> plot(ftrt)

```

It is very clear that there are periods of increased volatility, particularly around 2008-2009, late 2011 and more recently in mid 2015:

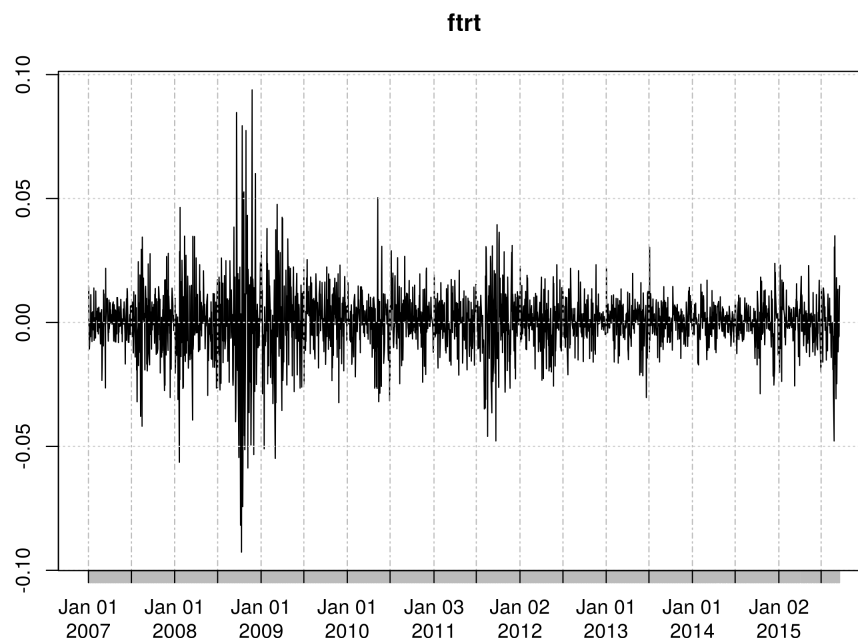


Figure 11.9: Differenced log returns of the daily closing price of the FTSE 100 UK stock index

We also need to remove the `NA` value generated by the differencing procedure:

```
> ft <- as.numeric(ftrt)
> ft <- ft[!is.na(ft)]
```

The next task is to fit a suitable ARIMA(p, d, q) model. We saw how to do that in the previous chapter so the procedure will not be repeated here. Instead the code will simply be provided:

```
> ftfinal.aic <- Inf
> ftfinal.order <- c(0,0,0)
> for (p in 1:4) for (d in 0:1) for (q in 1:4) {
>   ftcurent.aic <- AIC(arima(ft, order=c(p, d, q)))
>   if (ftcurent.aic < ftfinal.aic) {
>     ftfinal.aic <- ftcurent.aic
>     ftfinal.order <- c(p, d, q)
>     ftfinal.arima <- arima(ft, order=ftfinal.order)
>   }
> }
```

Since we have already differenced the FTSE returns once we should expect our integrated component d to equal zero, which it does:

```
> ftfinal.order
[1] 4 0 4
```

Thus we receive an ARIMA(4,0,4) model. This contains four autoregressive parameters and four moving average parameters.

We are now in a position to decide whether the *residuals* of this model fit possess evidence of conditional heteroskedastic behaviour. To test this we need to plot the correlogram of the residuals, as given in Figure [11.10](#)

```
> acf(resid(ftfinal.arima))
```

This looks like a realisation of a discrete white noise process indicating that we have achieved a good fit with the ARIMA(4,0,4) model.

To test for conditional heteroskedastic behaviour we need to square the residuals and plot the corresponding correlogram, as given in Figure [11.11](#)

```
> acf(resid(ftfinal.arima)^2)
```

We can see clear evidence of serial correlation in the squared residuals, leading us to the conclusion that conditional heteroskedastic behaviour is present in the diff log return series of the FTSE100.

We are *now* in a position to fit a GARCH model using the *tseries* library.

The first command actually fits an appropriate GARCH model, with the `trace=F` parameter telling R to suppress excessive output.

The second command removes the first element of the residuals, since it is NA:

```
> ft.garch <- garch(ft, trace=F)
> ft.res <- ft.garch$res[-1]
```

To test for a good fit we can plot the correlogram of the GARCH residuals and the square GARCH residuals as given in Figure [11.12](#)

```
> acf(ft.res)
```

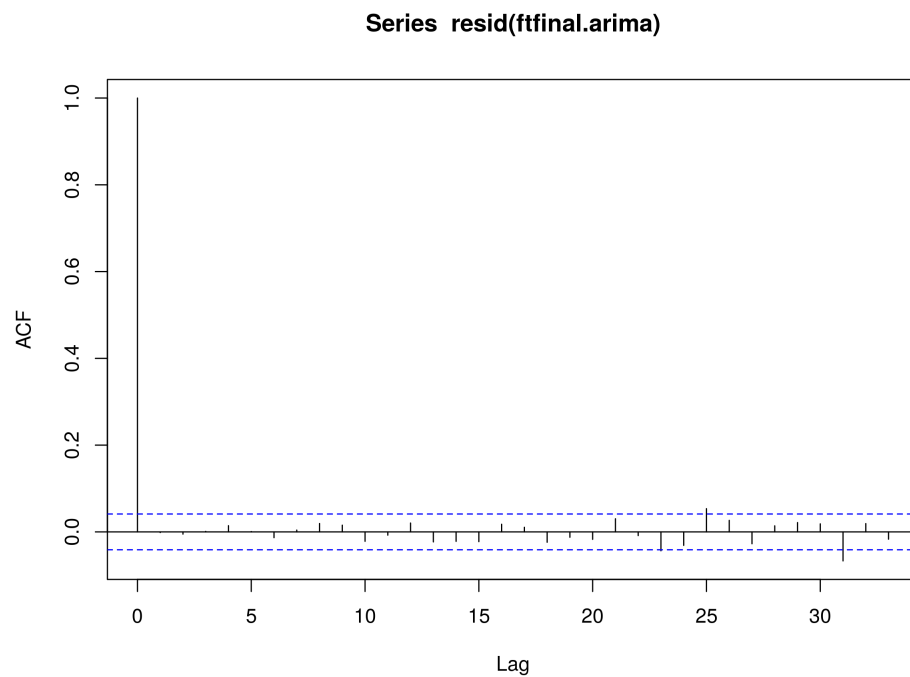



Figure 11.10: Residuals of an ARIMA(4,0,4) fit to the FTSE100 diff log returns

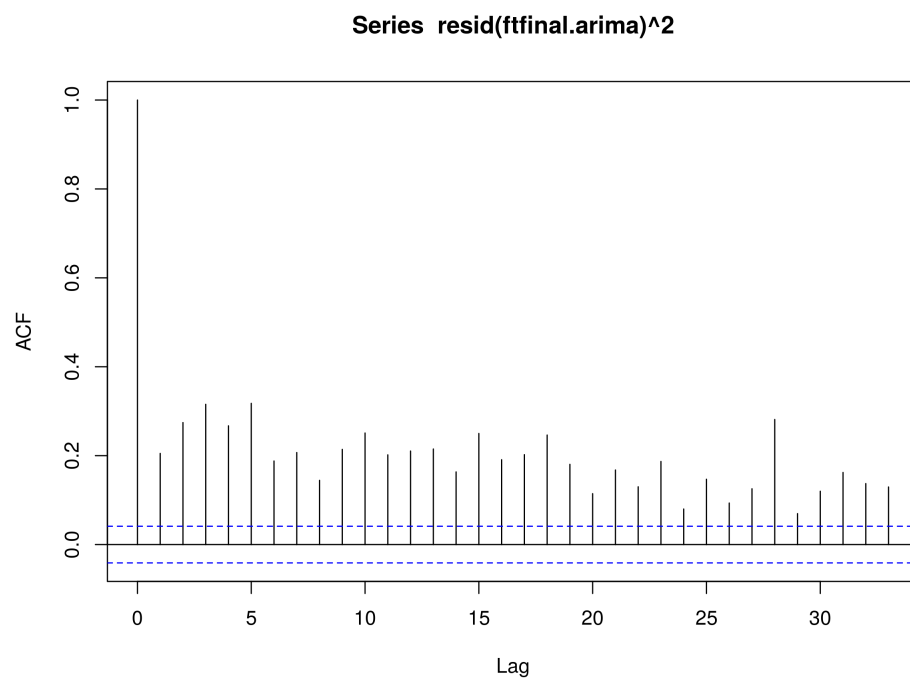


Figure 11.11: Squared residuals of an ARIMA(4,0,4) fit to the FTSE100 diff log returns

The correlogram looks like a realisation of a discrete white noise process indicating a good fit. Let us now try the squared residuals as given in Figure [11.13](#)

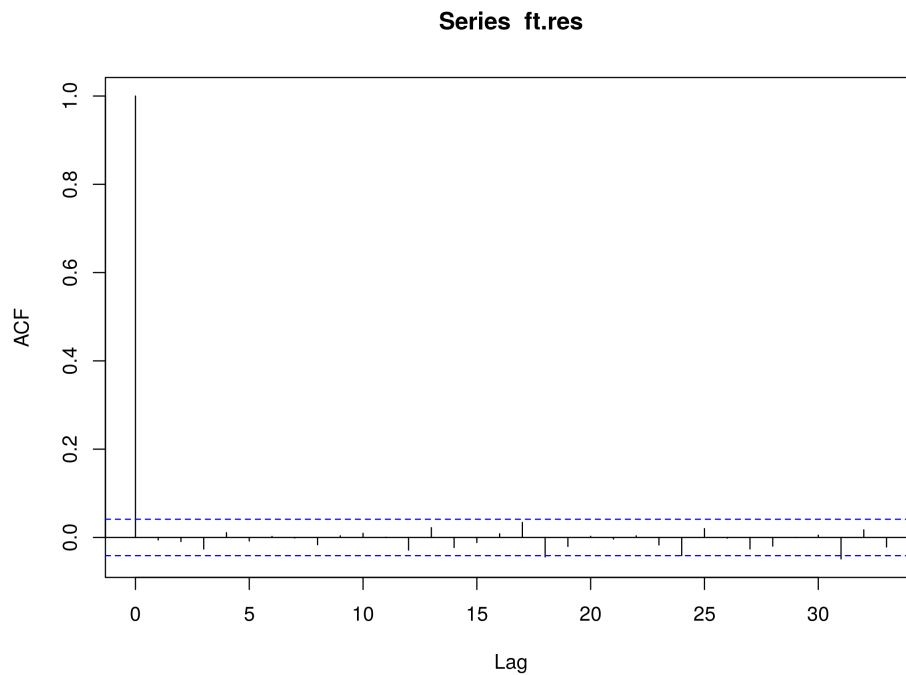


Figure 11.12: Residuals of a GARCH(p,q) fit to the ARIMA(4,0,4) fit of the FTSE100 diff log returns

```
> acf(ft.res^2)
```

Once again we have what looks like a realisation of a discrete white noise process, indicating that we have "explained" the serial correlation present in the squared residuals with an appropriate mixture of ARIMA(p,d,q) and GARCH(p,q).

11.7 Next Steps

We are now at the point in our time series education where we have studied ARIMA and GARCH. This knowledge allows us to fit a combination of these models to a stock market index and determine if we have achieved a good fit or not.

The next step is to actually produce forecasts of future daily returns values from this combination and use it to create a basic trading strategy. We will discuss this in the Quantitative Trading Strategies part of the book.

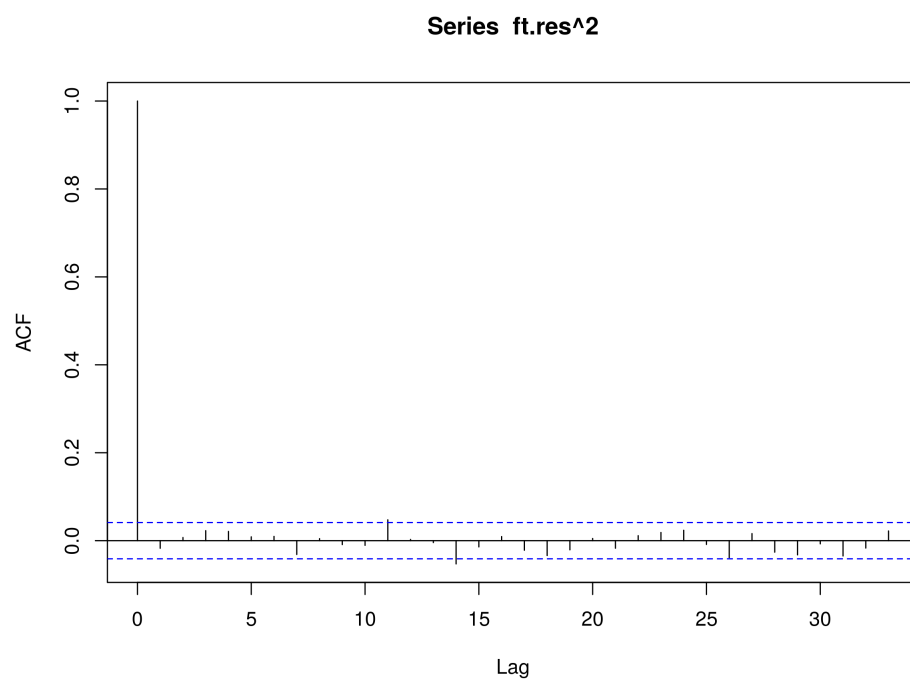


Figure 11.13: Squared residuals of a GARCH(p,q) fit to the ARIMA(4,0,4) fit of the FTSE100 diff log returns

Chapter 12

Cointegrated Time Series

In this chapter I want to discuss a topic called cointegration, which is a time series concept that allows us to determine if we are able to form a mean-reverting pair of assets. We will cover the time series theory related to cointegration here and in subsequent chapters we will show how to apply it to real trading strategies using the new open source backtesting framework: QSTrader.

We will proceed by discussing mean reversion in the traditional "pairs trading" framework. This will lead us to the concept of *stationarity* of a linear combination of assets, ultimately bringing us to *cointegration* and *unit root tests*. Once we have outlined these tests we will simulate various time series in the R statistical environment and apply the tests in order to assess cointegration.

12.1 Mean Reversion Trading Strategies

The traditional idea of a mean reverting "pairs trade" is to simultaneously long and short two separate assets sharing underlying factors that affect their movements. An example from the equities world might be to long McDonald's (NYSE:MCD) and short Burger King (NYSE:BKW - prior to the merger with Tim Horton's).

The rationale for this is that their long term share prices are likely to be in equilibrium due to the broad market factors affecting hamburger production and consumption. A short-term disruption to an individual in the pair, such as a supply chain disruption solely affecting McDonald's, would lead to a temporary dislocation in their relative prices. This means that a long-short trade carried out at this disruption point should become profitable as the two stocks return to their equilibrium value once the disruption is resolved. This is the essence of the classic "pairs trade".

As quants we are interested in carrying out mean reversion trading not solely on a *pair* of assets, but also *baskets* of assets that are separately interrelated.

To achieve this we need a robust mathematical framework for identifying pairs or baskets of assets that mean revert in the manner described above. This is where the concept of cointegrated time series arises.

The idea is to consider a pair of non-stationary time series, such as the (almost) random walk-like assets of MCD and BKW, and form a *linear combination* of each series to produce a stationary series, which has a fixed mean and variance.

This stationary series may have short term disruptions where the value wanders far from the

mean, but due to its stationarity this value will eventually return to the mean. Trading strategies can make use of this by longing/shorting the pair at the appropriate disruption point and betting on a longer-term reversion of the series to its mean.

Mean reverting strategies such as this permit a wide range of instruments to create the "synthetic" stationary time series. We are certainly not restricted to "vanilla" equities. For instance, we can make use of Exchange Traded Funds (ETF) that track commodity prices, such as crude oil, and baskets of oil producing companies. Hence there is plenty of scope for identifying such mean reverting systems.

Before we delve into the mechanics of the actual trading strategies, which will be the subject of subsequent chapters, we must first understand how to statistically identify such *cointegrated* series. For this we will utilise techniques from time series analysis, continuing the usage of the R statistical language as in previous chapters on the topic.

12.2 Cointegration

Now that we have motivated the necessity for a quantitative framework to carry out mean reversion trading we can define the concept of cointegration. Consider a pair of time series, both of which are non-stationary. If we take a particular linear combination of these series it can sometimes lead to a stationary series. Such a pair of series would then be termed *cointegrated*.

The mathematical definition is given by:

Definition 12.2.1. Cointegration. Let $\{x_t\}$ and $\{y_t\}$ be two non-stationary time series, with $a, b \in \mathbb{R}$, constants. If the combined series $ax_t + by_t$ is stationary then we say that $\{x_t\}$ and $\{y_t\}$ are **cointegrated**.

While the definition is useful it does not directly provide us with a mechanism for either determining the values of a and b , nor whether such a combination is in fact statistically stationary. For the latter we need to utilise tests for *unit roots*.

12.3 Unit Root Tests

In our previous discussion of autoregressive AR(p) models we explained the role of the *characteristic equation*. We noted that it was simply an autoregressive model, written in backward shift form, set to equal zero. Solving this equation gave us a set of *roots*.

In order for the model to be considered stationary all of the roots of the equation had to exceed unity. An AR(p) model with a root equal to unity—a *unit root*—is non-stationary. Random walks are AR(1) processes with unit roots and hence they are also non-stationary.

Thus in order to detect whether a time series is stationary or not we can construct a statistical hypothesis test for the presence of a unit root in a time series sample.

We are going to consider three separate tests for unit roots: Augmented Dickey-Fuller, Phillips-Perron and Phillips-Ouliaris. We will see that they are based on differing assumptions but are all ultimately testing for the same issue, namely stationarity of the tested time series sample.

Let us now take a brief look at all three tests in turn.

12.3.1 Augmented Dickey-Fuller Test

Dickey and Fuller [37] were responsible for introducing the following test for the presence of a unit root. The original test considers a time series $z_t = \alpha z_{t-1} + w_t$, in which w_t is discrete white noise. The null hypothesis is that $\alpha = 1$, while the alternative hypothesis is that $\alpha < 1$.

Said and Dickey [88] improved the original Dickey-Fuller test leading to the Augmented Dickey-Fuller (ADF) test, in which the series z_t is modified to an AR(p) model from an AR(1) model.

12.3.2 Phillips-Perron Test

The ADF test assumes an AR(p) model as an approximation for the time series sample and uses this to account for higher order autocorrelations. The Phillips-Perron test [79] does not assume an AR(p) model approximation. Instead a non-parametric kernel smoothing method is utilised on the stationary process w_t , which allows it to account for unspecified autocorrelation and heteroscedasticity.

12.3.3 Phillips-Ouliaris Test

The Phillips-Ouliaris test [78] is different from the previous two tests in that it is testing for evidence of cointegration among the residuals between two time series. The main idea here is that tests such as ADF, when applied to the estimated cointegrating residuals, do not have the Dickey-Fuller distributions under the null hypothesis where cointegration is not present. Instead, these distributions are known as Phillips-Ouliaris distributions and hence this test is more appropriate.

12.3.4 Difficulties with Unit Root Tests

While the ADF and Phillips-Perron test are equivalent asymptotically they can produce very different answers in finite samples [104]. This is because they handle autocorrelation and heteroscedasticity differently. It is necessary to be very clear which hypotheses are being tested for when applying these tests and not to blindly apply them to arbitrary series.

In addition unit root tests are not great at distinguishing highly persistent stationary processes from non-stationary processes. One must be very careful when using these on certain forms of financial time series. This can be especially problematic when the underlying relationship being modelled (i.e. mean reversion of two similar pairs) naturally breaks down due to regime change or other structural changes in the financial markets.

12.4 Simulated Cointegrated Time Series with R

Let us now apply the previous unit root tests to some simulated data that we know to be cointegrated. We can make use of the definition of cointegration to artificially create two non-stationary time series that share an underlying stochastic trend, but with a linear combination that is stationary.

Our first task is to define a random walk $z_t = z_{t-1} + w_t$, where w_t is discrete white noise. With the random walk z_t let us create two new time series x_t and y_t that both share the underlying stochastic trend from z_t , albeit by different amounts:

$$x_t = pz_t + w_{x,t} \quad (12.1)$$

$$y_t = qz_t + w_{y,t} \quad (12.2)$$

If we then take a linear combination $ax_t + by_t$:

$$ax_t + by_t = a(pz_t + w_{x,t}) + b(qz_t + w_{y,t}) \quad (12.3)$$

$$= (ap + bq)z_t + aw_{x,t} + bw_{y,t} \quad (12.4)$$

We see that we only achieve a stationary series (that is a combination of white noise terms) if $ap + bq = 0$. We can put some numbers to this to make it more concrete. Suppose $p = 0.3$ and $q = 0.6$. After some simple algebra we see that if $a = 2$ and $b = -1$ we have that $ap + bq = 0$, leading to a stationary series combination. Hence x_t and y_t are cointegrated when $a = 2$ and $b = -1$.

We can simulate this in R in order to visualise the stationary combination. Firstly, we wish to create and plot the underlying random walk series, z_t :

```
> set.seed(123)
> z <- rep(0, 1000)
> for (i in 2:1000) z[i] <- z[i-1] + rnorm(1)
> plot(z, type="l")
```

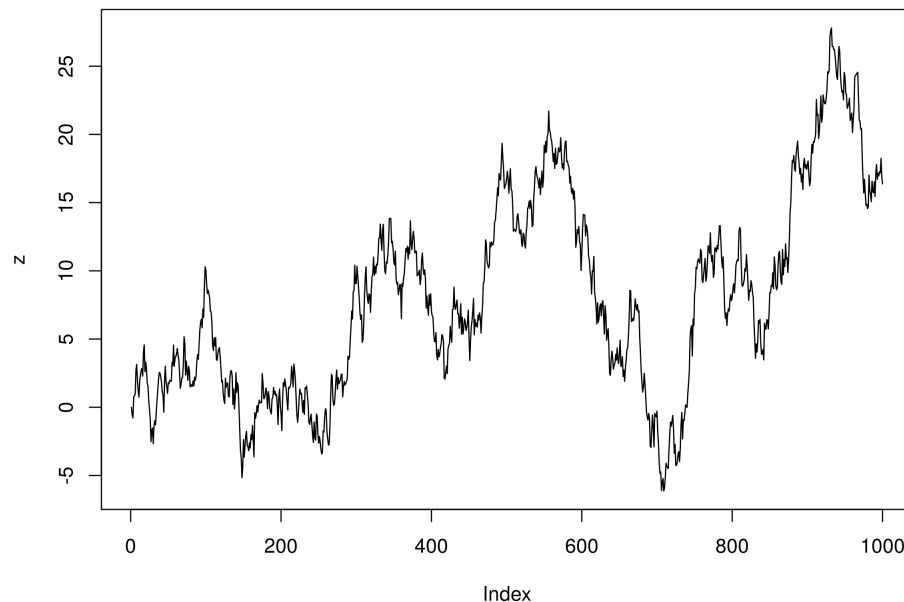


Figure 12.1: Realisation of a random walk, z_t

If we plot both the correlogram of the series and its differences we can see little evidence of

autocorrelation:

```
> layout(1:2)
> acf(z)
> acf(diff(z))
```

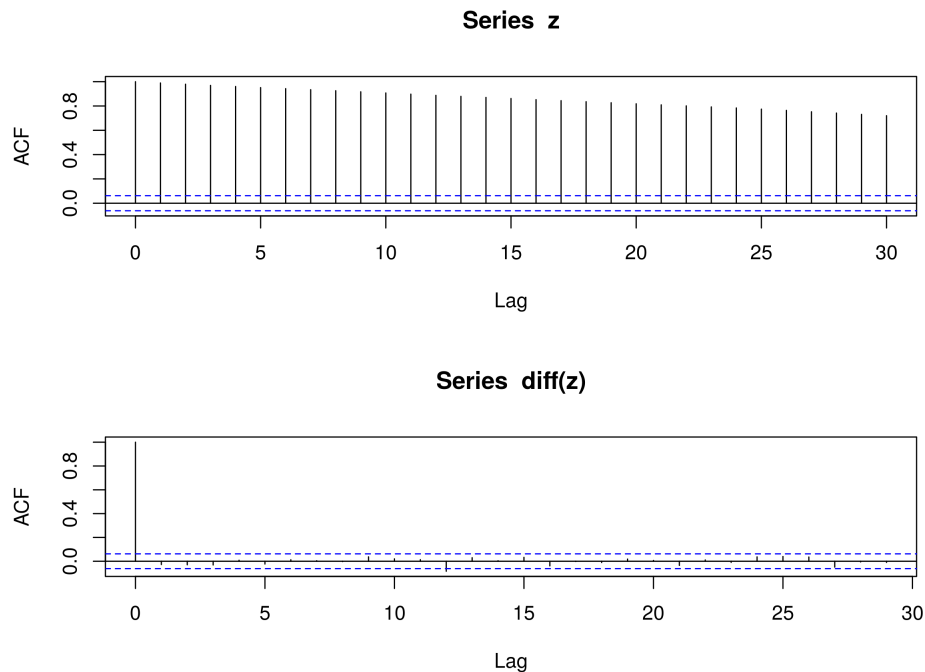


Figure 12.2: Correlograms of z_t and the differenced series of z_t

Hence this realisation of z_t clearly looks like a random walk. The next step is to create x_t and y_t from z_t , using $p = 0.3$ and $q = 0.6$, and then plot both:

```
> x <- y <- rep(0, 1000)
> x <- 0.3*z + rnorm(1000)
> y <- 0.6*z + rnorm(1000)
> layout(1:2)
> plot(x, type="l")
> plot(y, type="l")
```

As you can see they both look similar. Of course they will be by definition—they share the same underlying random walk structure from z_t . We can now form the linear combination, **comb**, using $p = 2$ and $q = -1$ and examine the autocorrelation structure:

```
> comb <- 2*x - y
> layout(1:2)
> plot(comb, type="l")
> acf(comb)
```

It is clear that the combination series **comb** looks very much like a stationary series. This is to be expected given its definition.

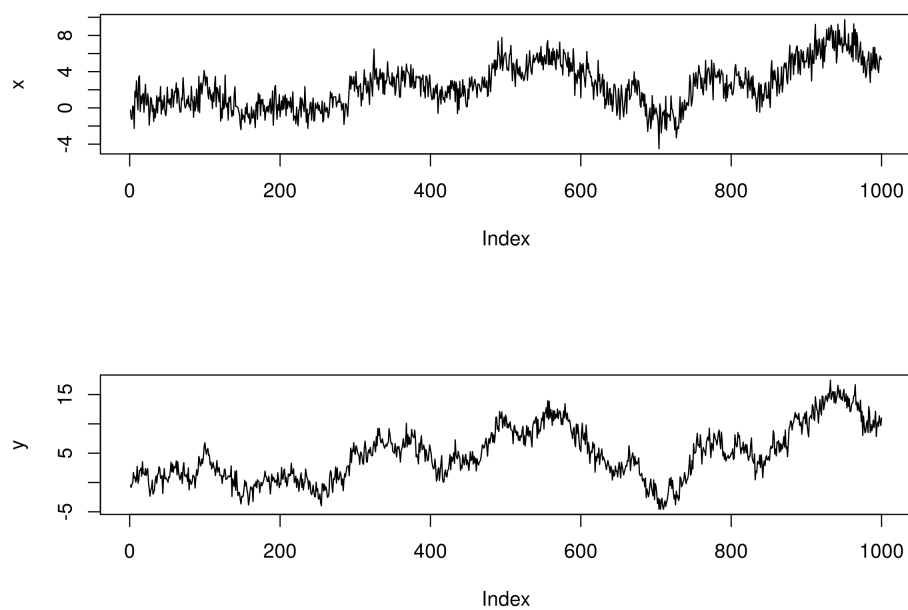


Figure 12.3: Plot of x_t and y_t series, each based on underlying random walk z_t

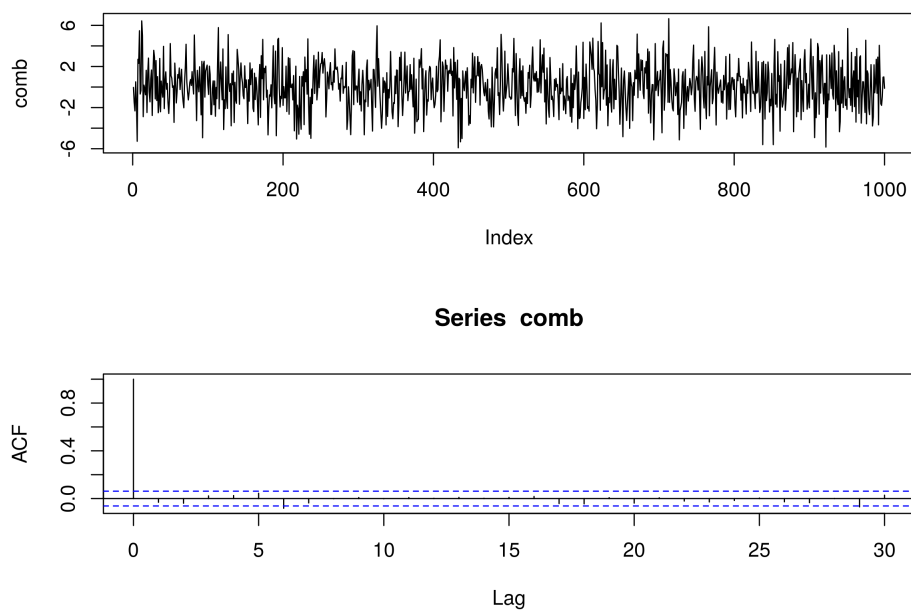


Figure 12.4: Plot of **comb** - the linear combination series - and its correlogram

Let us try applying the three unit root tests to the linear combination series. Firstly, the Augmented Dickey-Fuller test. For this and other tests it is necessary to import the **tseries**

library:

```
> library("tseries")
> adf.test(comb)
```

Augmented Dickey-Fuller Test

```
data:  comb
Dickey-Fuller = -10.321, Lag order = 9, p-value = 0.01
alternative hypothesis: stationary
```

Warning message:

```
In adf.test(comb) : p-value smaller than printed p-value
```

The p-value is small and hence we have evidence to reject the null hypothesis that the series possesses a unit root. Now we try the Phillips-Perron test:

```
> pp.test(comb)
```

Phillips-Perron Unit Root Test

```
data:  comb
Dickey-Fuller Z(alpha) = -1016.988, Truncation lag parameter = 7,
p-value = 0.01
alternative hypothesis: stationary
```

Warning message:

```
In pp.test(comb) : p-value smaller than printed p-value
```

Once again we have a small p-value and hence we have evidence to reject the null hypothesis of a unit root. Finally, we try the Phillips-Ouliaris test (notice that it requires matrix input of the underlying series constituents):

```
> po.test(cbind(2*x, -1.0*y))
```

Phillips-Ouliaris Cointegration Test

```
data:  cbind(2 * x, -1 * y)
Phillips-Ouliaris demeaned = -1023.784, Truncation lag parameter = 9,
p-value = 0.01
```

Warning message:

```
In po.test(cbind(2 * x, -1 * y)) : p-value smaller than printed p-value
```

Yet again we see a small p-value indicating evidence to reject the null hypothesis. Hence it is clear we are dealing with a pair of series that are cointegrated.

What happens if we instead create a separate combination with, say $p = -1$ and $q = 2$?

```
> badcomb <- -1.0*x + 2.0*y
```

```

> layout(1:2)
> plot(badcomb, type="l")
> acf(diff(badcomb))
> adf.test(badcomb)

```

Augmented Dickey-Fuller Test

```

data: badcomb
Dickey-Fuller = -2.4435, Lag order = 9, p-value = 0.3906
alternative hypothesis: stationary

```

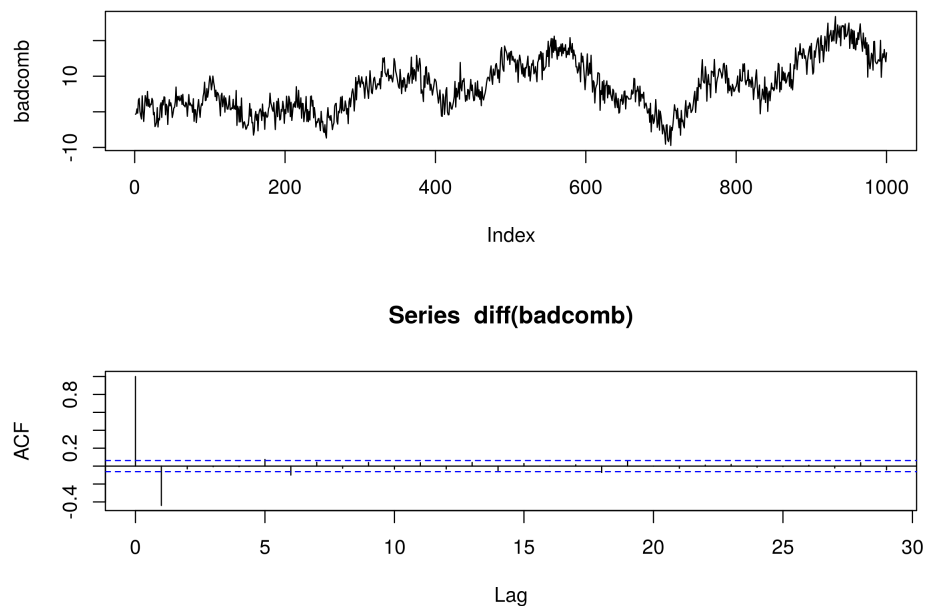


Figure 12.5: Plot of **badcomb**—the "incorrect" linear combination series—and its correlogram

In this case we do not have sufficient evidence to reject the null hypothesis of the presence of a unit root, as determined by p-value of the Augmented Dickey-Fuller test. This makes sense as we arbitrarily chose the linear combination of a and b rather than setting them to the correct values of $p = 2$ and $b = -1$ to form a stationary series.

12.5 Cointegrated Augmented Dickey Fuller Test

In the previous section we simulated two non-stationary time series that formed a cointegrated pair under a specific linear combination. We made use of the statistical Augmented Dickey-Fuller, Phillips-Perron and Phillips-Ouliaris tests for the presence of unit roots and cointegration.

A problem with the ADF test is that it does not provide us with the necessary β regression parameter—the hedge ratio—for forming the linear combination of the two time series. In this

section we are going to consider the Cointegrated Augmented Dickey-Fuller (CADF) procedure, which attempts to solve this problem.

While CADF will help us identify the β regression coefficient for our two series it will not tell us which of the two series is the dependent or independent variable for the regression. That is, the "response" value Y from the "feature" X , in statistical machine learning parlance. We will show how to avoid this problem by calculating the test statistic in the ADF test and using it to determine which of the two regressions will correctly produce a stationary series.

The main motivation for the CADF test is to determine an optimal hedging ratio to use between two pairs in a mean reversion trade, which was a problem that we identified with the analysis in the previous section. In essence it helps us determine how much of each pair to long and short when carrying out a pairs trade.

The CADF is a relatively simple procedure. We take a sample of historical data for two assets and then perform a linear regression between them, which produces α and β regression coefficients, representing the intercept and slope, respectively. The slope term helps us identify how much of each pair to relatively trade.

Once the slope coefficient—the hedge ratio—has been obtained we can then perform an ADF test on the linear regression residuals in order to determine evidence of stationarity and hence cointegration.

We will use R to carry out the CADF procedure, making use of the `tseries` and `quantmod` libraries for the ADF test and historical data acquisition, respectively.

We will begin by constructing a synthetic data set, with known cointegrating properties, to see if the CADF procedure can recover the stationarity and hedging ratio. We will then apply the same analysis to some real historical financial data as a precursor to implementing some mean reversion trading strategies.

12.6 CADF on Simulated Data

We are now going to demonstrate the CADF approach on simulated data. We will use the same simulated time series from the previous section.

Recall that we artificially created two non-stationary time series that formed a stationary residual series under a specific linear combination.

We can use the R linear model `lm` function to carry out a linear regression between the two series. This will provide us with an estimate for the regression coefficients and thus the optimal hedge ratio between the two series.

We begin by importing the `tseries` library, necessary for the ADF test:

```
> library("tseries")
```

Since we wish to use the same underlying stochastic trend series as in the previous section we set the seed for the random number generator as before:

```
> set.seed(123)
```

In the previous section we created an underlying stochastic random walk time series, z_t :

```
> z <- rep(0, 1000)
> for (i in 2:1000) z[i] <- z[i-1] + rnorm(1)
```

We then created two subsequent series, which I will rename to p_t and q_t so that we do not confuse the original names y_t and x_t with the conventional names for regression responses and predictors:

```
> p <- q <- rep(0, 1000)
> p <- 0.3*z + rnorm(1000)
> q <- 0.6*z + rnorm(1000)
```

At this stage we can make use of the `lm` function, which calculates a linear regression between two vectors. In this instance we will set q_t to be the independent variable and p_t to be the dependent variable:

```
> comb <- lm(p~q)
```

If we take a look at the `comb` linear regression model we can see that the estimate for the β regression coefficient is approximately 0.5, which makes sense given that q_t is twice dependent on z_t compared to p_t (0.6 compared to 0.3):

```
> comb
```

Call:

```
lm(formula = p ~ q)
```

Coefficients:

(Intercept)	q
0.1749	0.4745

Finally, we apply the ADF test to the residuals of the linear model in order to test for stationarity:

```
> adf.test(comb$residuals, k=1)
```

Augmented Dickey-Fuller Test

data: comb\$residuals

Dickey-Fuller = -23.4463, Lag order = 1, p-value = 0.01

alternative hypothesis: stationary

Warning message:

In adf.test(comb\$residuals, k = 1) : p-value smaller than printed p-value

The Dickey-Fuller test statistic is very low, providing us with a low p-value. We can likely reject the null hypothesis of the presence of a unit root and conclude that we have a stationary series and hence a cointegrated pair. This is clearly not surprising given that we simulated the data to have these properties in the first place.

We are now going to apply the CADF procedure to multiple sets of historical financial data.

12.7 CADF on Financial Data

There are many ways of forming a cointegrating set of assets. A common source is to use ETFs that track similar characteristics. A good example is an ETF representing a basket of gold mining

firms paired with an ETF that tracks the spot price of gold. Similarly for crude oil or any other commodity.

An alternative is to form tighter cointegrating pairs by considering separate share classes on the same stock, as with the Royal Dutch Shell example below. Another is the famous Berkshire Hathaway holding company, run by Warren Buffet and Charlie Munger, which also has A shares and B shares. However, in this instance we need to be careful because we must ask ourselves whether we would likely be able to form a profitable mean reversion trading strategy on such a pair, given how tight the cointegration is likely to be.

12.7.1 EWA and EWC

A famous example in the quant community of the CADF test applied to equities data is given by Ernie Chan [\[32\]](#). He forms a cointegrating pair from two ETFs, with ticker symbols EWA and EWC, representing a set of Australian and Canadian equities baskets, respectively. The logic is that both of these countries are heavily commodities based and so will likely have a similar underlying stochastic trend.

Ernie makes use of MatLab for his work, but this section is concentrating on R. Hence I thought it would be instructive to utilise the same starting and ending dates for his historical analysis in order to see how the results compare.

The first task is to import the R quant finance library, `quantmod`, which will be helpful for us in downloading financial data:

```
> library("quantmod")
```

We then need to obtain the backward-adjusted closing prices from Yahoo Finance for EWA and EWC across the exact period used in Ernie's work - April 26th 2006 to April 9th 2012:

```
> getSymbols("EWA", from="2006-04-26", to="2012-04-09")
> getSymbols("EWC", from="2006-04-26", to="2012-04-09")
```

We now place the adjusted prices into the `ewaAdj` and `ewcAdj` variables:

```
> ewaAdj = unclass(EWA$EWA.Adjusted)
> ewcAdj = unclass(EWC$EWC.Adjusted)
```

For completeness I will replicate the plots from Ernie's work in order that you can see the same code in the R environment. Firstly let us plot the adjusted ETF prices themselves.

To carry this out in R we need to utilise the `par(new=T)` command to append to a plot rather than renew it. Notice that I have set the axes on both `plot(...)` commands to be equal and have not added axes to the second plot. If we do not do this the plot becomes cluttered and illegible:

```
> plot(ewaAdj, type="l", xlim=c(0, 1500), ylim=c(5.0, 35.0),
      xlab="April 26th 2006 to April 9th 2012",
      ylab="ETF Backward-Adjusted Price in USD", col="blue")
> par(new=T)
> plot(ewcAdj, type="l", xlim=c(0, 1500), ylim=c(5.0, 35.0),
      axes=F, xlab="", ylab="", col="red")
> par(new=F)
```

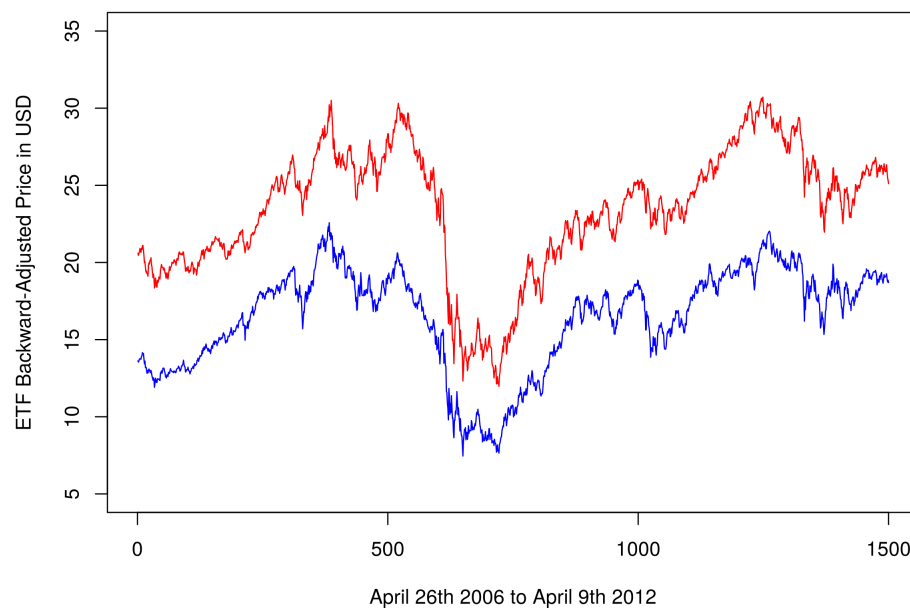


Figure 12.6: Backward-adjusted closing prices of EWA and EWC

You will notice that it differs slightly from the chart given in Ernie's work as we are plotting the adjusted prices here, rather than the unadjusted closing prices. We can also create a scatter plot of their prices:

```
> plot(ewaAdj, ewcAdj, xlab="EWA Backward-Adjusted Prices",
      ylab="EWC Backward-Adjusted Prices")
```

At this stage we need to perform the linear regressions between the two price series. However, we have previously mentioned that it is unclear as to which series is the dependent variable and which is the independent variable for the regression. Thus we will try both and make a choice based on the negativity of the ADF test statistic. We will use the R linear model (`lm`) function for the regression:

```
> comb1 = lm(ewcAdj~ewaAdj)
> comb2 = lm(ewaAdj~ewcAdj)
```

This will provide us with the intercept and regression coefficient for these pairs. We can plot the residuals and visually assess the stationarity of the series:

```
> plot(comb1$residuals, type="l",
      xlab="April 26th 2006 to April 9th 2012",
      ylab="Residuals of EWA and EWC regression")
```

We can also view the regression coefficients starting with EWA as the independent variable:

```
> comb1
```

Call:

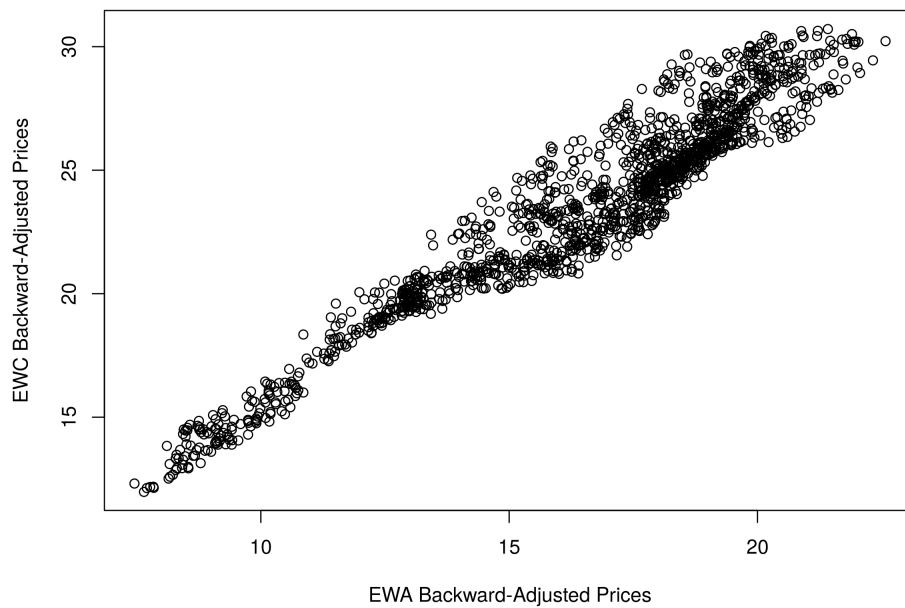


Figure 12.7: Scatter plot of backward-adjusted closing prices for EWA and EWC

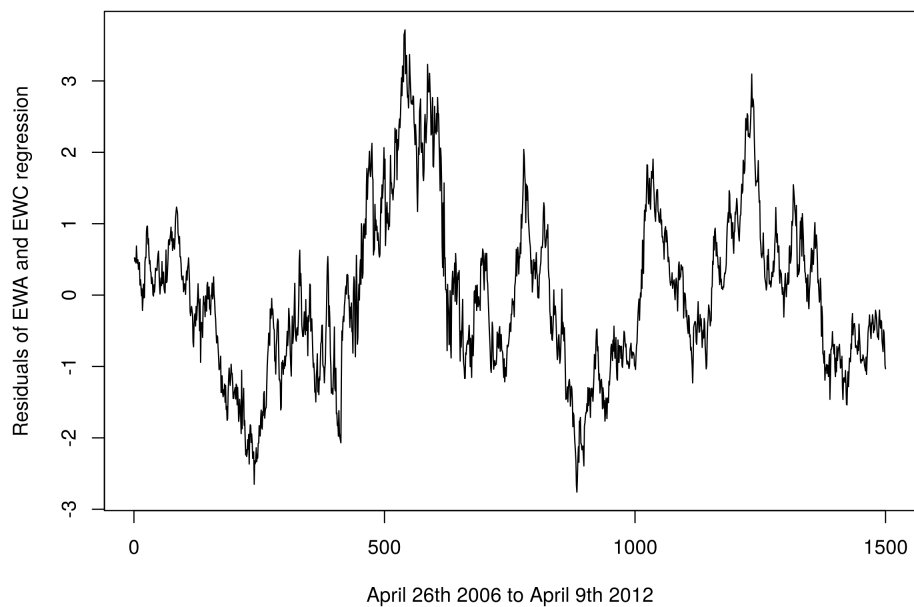


Figure 12.8: Residuals of the first linear combination of EWA and EWC

```
lm(formula = ewcAdj ~ ewaAdj)
```


Coefficients:

(Intercept)	ewaAdj
3.809	1.194

This provides us with an intercept of $\alpha = 3.809$ and a $\beta = 1.194$. Similarly for EWC as the independent variable:

```
> comb2
```

Call:

```
lm(formula = ewaAdj ~ ewcAdj)
```

Coefficients:

(Intercept)	ewcAdj
-1.638	0.771

This provides us with an intercept $\alpha = -1.638$ and a $\beta = 0.771$. The key issue here is that they are not equal to the previous regression coefficients. Hence we must use the ADF test statistic in order to determine the optimal hedge ratio. For EWA as the independent variable:

```
> adf.test(comb1$residuals, k=1)
```

Augmented Dickey-Fuller Test

data: comb1\$residuals

Dickey-Fuller = -3.6357, Lag order = 1, p-value = 0.02924

alternative hypothesis: stationary

Our test statistic gives a p-value less than 0.05 providing evidence that we can reject the null hypothesis of a unit root at the 5% level. Similarly for EWC as the independent variable:

```
> adf.test(comb2$residuals, k=1)
```

Augmented Dickey-Fuller Test

data: comb2\$residuals

Dickey-Fuller = -3.6457, Lag order = 1, p-value = 0.02828

alternative hypothesis: stationary

Once again we have evidence to reject the null hypothesis of the presence of a unit root, leading to evidence for a stationary series (and cointegrated pair) at the 5% level.

The ADF test statistic for EWC as the independent variable is smaller (more negative) than that for EWA as the independent variable and hence we will choose this as our linear combination for any future trading implementations.

12.7.2 RDS-A and RDS-B

A common method of obtaining a strong cointegrated relationship is to take two publicly traded share classes of the same underlying equity. One such pair is given by the London-listed Royal Dutch Shell oil major, with its two share classes RDS-A and RDS-B.

We can replicate the above steps for RDS-A and RDS-B as we did for EWA and EWC. The full code to carry this out is given below. The only minor difference is that we need to utilise the `get(...)` R function, since quantmod pulls in RDS-A as the variable "RDS-A". R does not like hyphens in variable names as the minus operator takes precedence. Hence we need to use `get` as a workaround:

```
> getSymbols("RDS-A", from="2006-01-01", to="2015-12-31")
> getSymbols("RDS-B", from="2006-01-01", to="2015-12-31")
> RDSA <- get("RDS-A")
> RDSB <- get("RDS-B")
> rdsaAdj = unclass(RDSA$"RDS-A.Adjusted")
> rdsbAdj = unclass(RDSB$"RDS-B.Adjusted")
```

We can plot both share classes on the same chart. Clearly they are tightly cointegrated:

```
> plot(rdsaAdj, type="l", xlim=c(0, 2517), ylim=c(25.0, 80.0),
      xlab="January 1st 2006 to December 31st 2015",
      ylab="RDS-A and RDS-B Backward-Adjusted Closing Price in GBP", col="blue")
> par(new=T)
> plot(rdsbAdj, type="l", xlim=c(0, 2517), ylim=c(25.0, 80.0), axes=F,
      xlab="", ylab="", col="red")
> par(new=F)
```

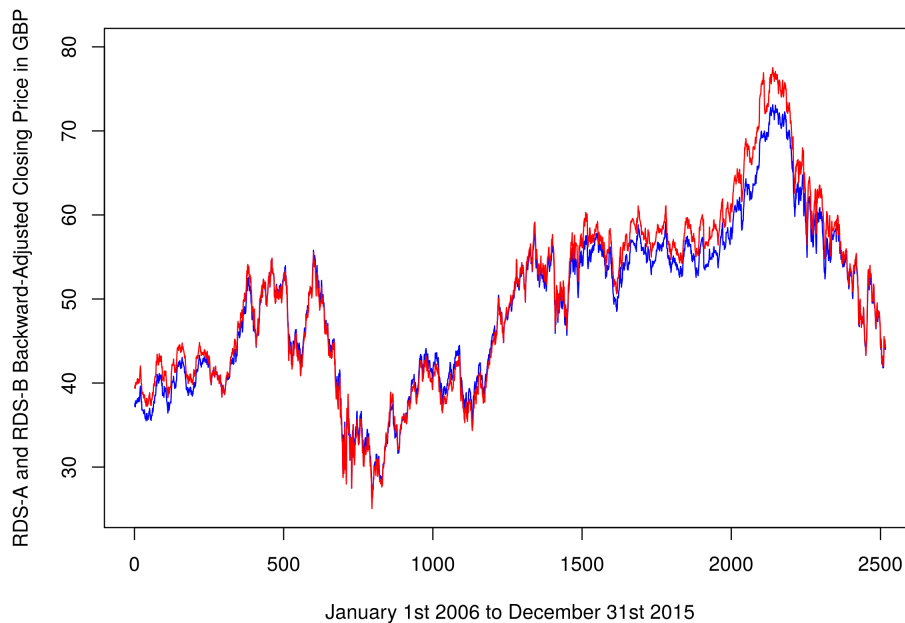


Figure 12.9: Backward-adjusted closing prices of RDS-A and RDS-B

We can also plot a scatter graph of the two price series. It is apparent how tight the linear relationship between them is. This is no surprise given that they track the same underlying equity:

```
> plot(rdsaAdj, rdsbAdj,
      xlab="RDS-A Backward-Adjusted Prices",
      ylab="RDS-B Backward-Adjusted Prices")
```

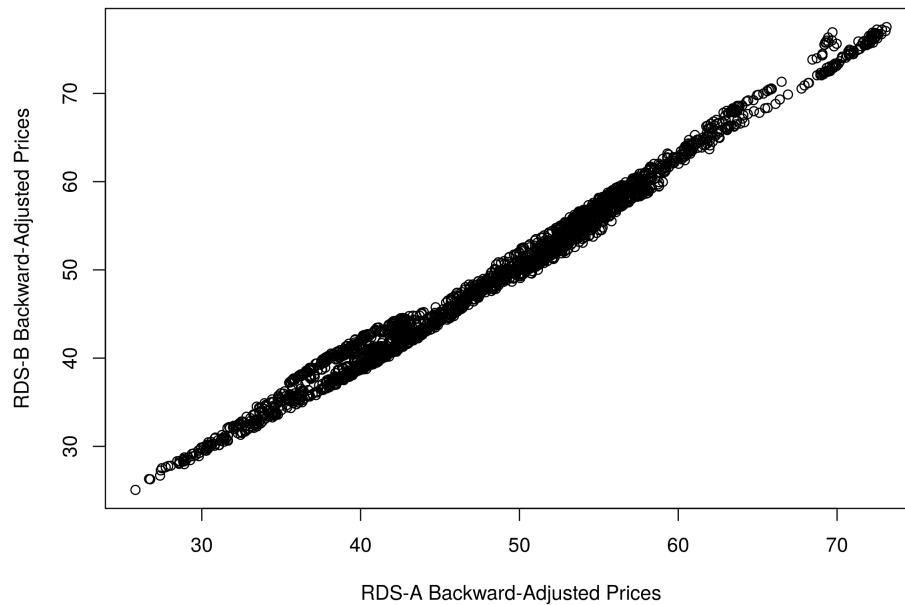


Figure 12.10: Scatter plot of backward-adjusted closing prices for RDS-A and RDS-B

Once again we utilise the linear model `lm` function to ascertain the regression coefficients, making sure to swap the dependent and independent variables for the second regression. We can then plot the residuals of the first regression:

```
> comb1 = lm(rdsaAdj~rdsbAdj)
> comb2 = lm(rdsbAdj~rdsaAdj)
> plot(comb1$residuals, type="l",
      xlab="January 1st 2006 to December 31st 2015",
      ylab="Residuals of RDS-A and RDS-B regression")
```

Finally, we can calculate the ADF test-statistic to ascertain the optimal hedge ratio. For the first linear combination:

```
> adf.test(comb1$residuals, k=1)
```

Augmented Dickey-Fuller Test

data: comb1\$residuals

Dickey-Fuller = -4.0537, Lag order = 1, p-value = 0.01

alternative hypothesis: stationary

Warning message:

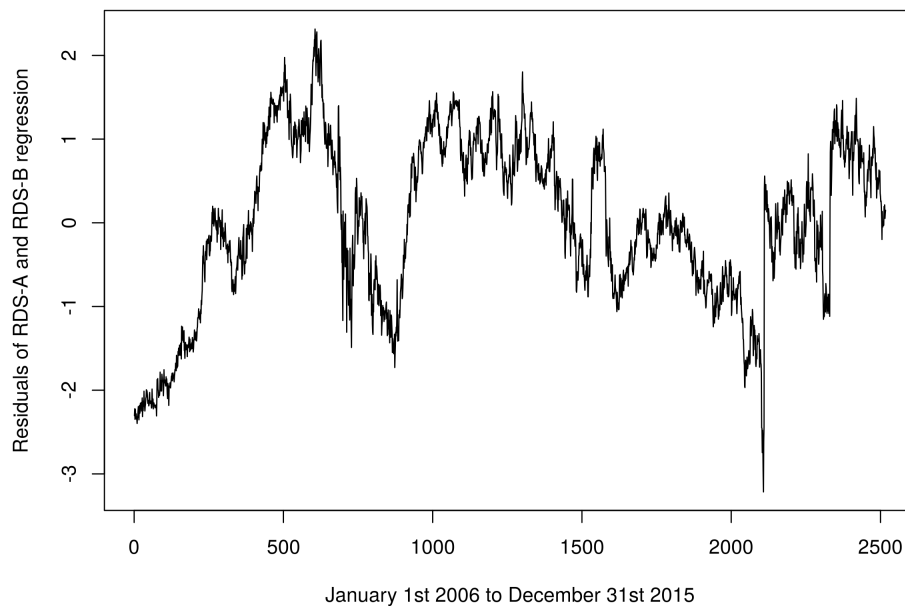


Figure 12.11: Residuals of the first linear combination of RDS-A and RDS-B

```
In adf.test(comb1$residuals, k = 1) : p-value smaller than printed p-value
```

And for the second:

```
> adf.test(comb2$residuals, k=1)
```

Augmented Dickey-Fuller Test

data: comb2\$residuals

Dickey-Fuller = -3.9846, Lag order = 1, p-value = 0.01

alternative hypothesis: stationary

Warning message:

```
In adf.test(comb2$residuals, k = 1) : p-value smaller than printed p-value
```

Since the first linear combination has the smallest Dickey-Fuller statistic, we conclude that this is the optimal linear regression. In any subsequent trading strategy we would utilise these regression coefficients for our relative long-short positioning.

12.8 Full Code

```
library("quantmod")
library("tseries")

## Set the random seed to 123
```

```

set.seed(123)

## SIMULATED DATA

## Create a simulated random walk
z <- rep(0, 1000)
for (i in 2:1000) z[i] <- z[i-1] + rnorm(1)

## Create two non-stationary series based on the
## simulated random walk
p <- q <- rep(0, 1000)
p <- 0.3*z + rnorm(1000)
q <- 0.6*z + rnorm(1000)

## Perform a linear regression against the two
## simulated series in order to assess the hedge ratio
comb <- lm(p~q)

## FINANCIAL DATA - EWA/EWC

## Obtain EWA and EWC for dates corresponding to Chan (2013)
getSymbols("EWA", from="2006-04-26", to="2012-04-09")
getSymbols("EWC", from="2006-04-26", to="2012-04-09")

## Utilise the backwards-adjusted closing prices
ewaAdj = unclass(EWA$EWA.Adjusted)
ewcAdj = unclass(EWC$EWC.Adjusted)

## Plot the ETF backward-adjusted closing prices
plot(ewaAdj, type="l", xlim=c(0, 1500), ylim=c(5.0, 35.0),
     xlab="April 26th 2006 to April 9th 2012",
     ylab="ETF Backward-Adjusted Price in USD", col="blue")
par(new=T)
plot(ewcAdj, type="l", xlim=c(0, 1500), ylim=c(5.0, 35.0),
     axes=F, xlab="", ylab="", col="red")
par(new=F)

## Plot a scatter graph of the ETF adjusted prices
plot(ewaAdj, ewcAdj, xlab="EWA Backward-Adjusted Prices",
     ylab="EWC Backward-Adjusted Prices")

## Carry out linear regressions twice, swapping the dependent
## and independent variables each time, with zero drift
comb1 = lm(ewcAdj~ewaAdj)
comb2 = lm(ewaAdj~ewcAdj)

```

```

## Plot the residuals of the first linear combination
plot(comb1$residuals, type="l",
      xlab="April 26th 2006 to April 9th 2012",
      ylab="Residuals of EWA and EWC regression")

## Now we perform the ADF test on the residuals,
## or "spread" of each model, using a single lag order
adf.test(comb1$residuals, k=1)
adf.test(comb2$residuals, k=1)

## FINANCIAL DATA - RDS-A/RDS-B

## Obtain RDS equities prices for a recent ten year period
getSymbols("RDS-A", from="2006-01-01", to="2015-12-31")
getSymbols("RDS-B", from="2006-01-01", to="2015-12-31")

## Avoid the hyphen in the name of each variable
RDSA <- get("RDS-A")
RDSB <- get("RDS-B")

## Utilise the backwards-adjusted closing prices
rdsaAdj = unclass(RDSA$"RDS-A.Adjusted")
rdsbAdj = unclass(RDSB$"RDS-B.Adjusted")

## Plot the ETF backward-adjusted closing prices
plot(rdsaAdj, type="l", xlim=c(0, 2517), ylim=c(25.0, 80.0),
      xlab="January 1st 2006 to December 31st 2015",
      ylab="RDS-A and RDS-B Backward-Adjusted Closing Price in GBP", col="blue")
par(new=T)
plot(rdsbAdj, type="l", xlim=c(0, 2517), ylim=c(25.0, 80.0),
      axes=F, xlab="", ylab="", col="red")
par(new=F)

## Plot a scatter graph of the
## Royal Dutch Shell adjusted prices
plot(rdsaAdj, rdsbAdj, xlab="RDS-A Backward-Adjusted Prices",
      ylab="RDS-B Backward-Adjusted Prices")

## Carry out linear regressions twice, swapping the dependent
## and independent variables each time, with zero drift
comb1 = lm(rdsaAdj~rdsbAdj)
comb2 = lm(rdsbAdj~rdsaAdj)

## Plot the residuals of the first linear combination

```

```

plot(comb1$residuals, type="l",
     xlab="January 1st 2006 to December 31st 2015",
     ylab="Residuals of RDS-A and RDS-B regression")

## Now we perform the ADF test on the residuals,
## or "spread" of each model, using a single lag order
adf.test(comb1$residuals, k=1)
adf.test(comb2$residuals, k=1)

```

12.9 Johansen Test

In the previous section on the Cointegrated Augmented Dickey Fuller (CADF) test we noted that one of the biggest drawbacks of the test was that it was only capable of being applied to two separate time series. However, we can clearly imagine a set of three or more financial assets that might share an underlying cointegrated relationship.

A trivial example would be three separate share classes on the same asset, while a more interesting example would be three separate ETFs that all track certain areas of commodity equities and the underlying commodity spot prices.

In this section we are going to discuss a test due to Johansen^[60] that allows us to determine if three or more time series are cointegrated. We will then form a stationary series by taking a linear combination of the underlying series. Such a procedure will be utilised in subsequent chapters to form a mean reverting portfolio of assets for trading purposes.

We will begin by describing the theory underlying the Johansen test and then perform our usual procedure of carrying out the test on simulated data with known cointegrating properties. Subsequently we will apply the test to historical financial data and see if we can find a portfolio of cointegrated assets.

We will now outline the mathematical underpinnings of the Johansen procedure, which allows us to analyse whether two or more time series can form a cointegrating relationship. In quantitative trading this would allow us to form a portfolio of two or more securities in a mean reversion trading strategy.

The theoretical details of the Johansen test require a bit of experience with multivariate time series, which we have not considered in the book as of yet. In particular we need to consider Vector Autoregressive Models (VAR)—not to be confused with Value at Risk (VaR)—which are a multidimensional extension of the Autoregressive Models studied in previous chapters.

A general vector autoregressive model is similar to the AR(p) model except that each quantity is vector-valued with matrices as the coefficients. The general form of the VAR(p) model, without drift, is given by:

$$\mathbf{x}_t = \mu + A_1 \mathbf{x}_{t-1} + \dots + A_p \mathbf{x}_{t-p} + \mathbf{w}_t \quad (12.5)$$

Where μ is the vector-valued mean of the series, A_i are the coefficient matrices for each lag and $\mathbf{w}_t$ is a multivariate Gaussian noise term with mean zero.

At this stage we can form what is known as a Vector Error Correction Model (VECM):

$$\Delta \mathbf{x}_t = \mu + A\mathbf{x}_{t-1} + \Gamma_1 \Delta \mathbf{x}_{t-1} + \dots + \Gamma_p \Delta \mathbf{x}_{t-p} + \mathbf{w}_t \quad (12.6)$$

Where $\Delta \mathbf{x}_t := \mathbf{x}_t - \mathbf{x}_{t-1}$ is the differencing operator, A is the coefficient matrix for the first lag and Γ_i are the matrices for each differenced lag.

The test checks for the situation of no cointegration, which occurs when the matrix $A = 0$.

The Johansen test is more flexible than the CADF procedure outlined in the previous section and can check for multiple linear combinations of time series for forming stationary portfolios.

To achieve this an *eigenvalue decomposition* of A is carried out. The rank of the matrix A is given by r and the Johansen test *sequentially* tests whether this rank r is equal to zero, equal to one, through to $r = n - 1$, where n is the number of time series under test.

The null hypothesis of $r = 0$ means that there is no cointegration at all. A rank $r > 0$ implies a cointegrating relationship between two or possibly more time series.

The eigenvalue decomposition results in a set of eigenvectors. The components of the largest eigenvector admits the important property of forming the coefficients of a linear combination of time series to produce a stationary portfolio. Notice how this differs from the CADF test (often known as the Engle-Granger procedure) where it is necessary to ascertain the linear combination a priori via linear regression and ordinary least squares (OLS).

In the Johansen test the linear combination values are estimated *as part of the test*, which implies that there is less statistical power associated with the test when compared to CADF. It is possible to run into situations where there is insufficient evidence to reject the null hypothesis of no cointegration despite the CADF suggesting otherwise. This will be discussed further below.

Perhaps the best way to understand the Johansen test is to see it applied to both simulated and historical financial data.

12.9.1 Johansen Test on Simulated Data

Now that we have outlined the theory of the test we are going to apply it using the R statistical environment. We will make use of the `urca` library, written by Bernhard Pfaff and Matthieu Stigler, which wraps up the Johansen test in an easy to call function - `ca.jo`.

The first task is to import the `urca` library itself:

```
> library("urca")
```

As in the previous cointegration sections we set the seed so that the results of the random number generator can be replicated in other R environments:

```
> set.seed(123)
```

We then create the underlying random walk z_t as in previous sections:

```
> z <- rep(0, 10000)
> for (i in 2:10000) z[i] <- z[i-1] + rnorm(1)
```

We create three time series that share the underlying random walk structure from z_t . They are denoted by p_t , q_t and r_t , respectively:

```
> p <- q <- r <- rep(0, 10000)
> p <- 0.3*z + rnorm(10000)
```



```
> q <- 0.6*z + rnorm(10000)
> r <- 0.2*z + rnorm(10000)
```

We then call the `ca.jo` function applied to a data frame of all three time series.

The `type` parameter tells the function whether to use the trace test statistic or the maximum eigenvalue test statistic, which are the two separate forms of the Johansen test. In this instance we are using `trace`.

`K` is the number of lags to use in the vector autoregressive model and is set this to the minimum, `K=2`.

`ecdet` refers to whether to use a constant or drift term in the model, while `spec="longrun"` refers to the specification of the VECM discussed above. This parameter can be `spec="longrun"` or `spec="transitory"`.

Finally, we print the summary of the output:

```
> jotest=ca.jo(data.frame(p,q,r), type="trace", K=2, ecdet="none",
  spec="longrun")
> summary(jotest)
```

```
#####
# Johansen-Procedure #
#####
```

Test type: trace statistic , with linear trend

Eigenvalues (**lambda**):

```
[1] 0.338903321 0.330365610 0.001431603
```

Values of teststatistic **and** critical values of test:

	test	10pct	5pct	1pct
$r \leq 2$	14.32	6.50	8.18	11.65
$r \leq 1$	4023.76	15.66	17.95	23.52
$r = 0$	8161.48	28.71	31.52	37.22

Eigenvectors, normalised to first column:

(These are the cointegration relations)

	p.l2	q.l2	r.l2
p.l2	1.000000	1.00000000	1.000000
q.l2	1.791324	-0.52269002	1.941449
r.l2	-1.717271	0.01589134	2.750312

Weights W:

(This **is** the loading matrix)

	p.l2	q.l2	r.l2
--	------	------	------

```
p.d -0.1381095 -0.771055116 -0.0003442470
q.d -0.2615348  0.404161806 -0.0006863351
r.d  0.2439540 -0.006556227 -0.0009068179
```

Let us try and interpret all of this information! The first section shows the eigenvalues generated by the test. In this instance we have three with the largest approximately equal to 0.3389.

The next section shows the trace test statistic for the three hypotheses of $r \leq 2$, $r \leq 1$ and $r = 0$. For each of these three tests we have not only the statistic itself (given under the **test** column) but also the critical values at certain levels of confidence: 10%, 5% and 1% respectively.

The first hypothesis, $r = 0$, tests for the presence of cointegration. It is clear that since the test statistic exceeds the 1% level significantly ($8161.48 > 37.22$) that we have strong evidence to reject the null hypothesis of no cointegration. The second test for $r \leq 1$ against the alternative hypothesis of $r > 1$ also provides clear evidence to reject $r \leq 1$ since the test statistic exceeds the 1% level significantly. The final test for $r \leq 2$ against $r > 2$ also provides sufficient evidence for rejecting the null hypothesis that $r \leq 2$ and so can conclude that the rank of the matrix r is greater than 2.

Thus the best estimate of the rank of the matrix is $r = 3$, which tells us that we need a linear combination of three time series to form a stationary series. This is to be expected, by definition of the series, as the underlying random walk utilised for all three series is non-stationary.

How do we go about forming such a linear combination? The answer is to make use of the eigenvector components of the eigenvector associated with the largest eigenvalue. We previously mentioned that the largest eigenvalue is approximately 0.3389. It corresponds to the vector given under the column **p.l2**, and is approximately equal to $(1.000000, 1.791324, -1.717271)$. If we form a linear combination of series using these components, we will receive a stationary series:

```
> s = 1.000*p + 1.791324*q - 1.717271*r
> plot(s, type="l")
```

Visually this looks very much like a stationary series. We can import the Augmented Dickey-Fuller test as an additional check:

```
> library("tseries")
> adf.test(s)

Augmented Dickey-Fuller Test

data:  s
Dickey-Fuller = -22.0445, Lag order = 21, p-value = 0.01
alternative hypothesis: stationary

Warning message:
In adf.test(s) : p-value smaller than printed p-value
```

The Dickey-Fuller test statistic is very low, providing a low p-value and hence evidence to reject the null hypothesis of a unit root and thus evidence we have a stationary series formed from a linear combination.

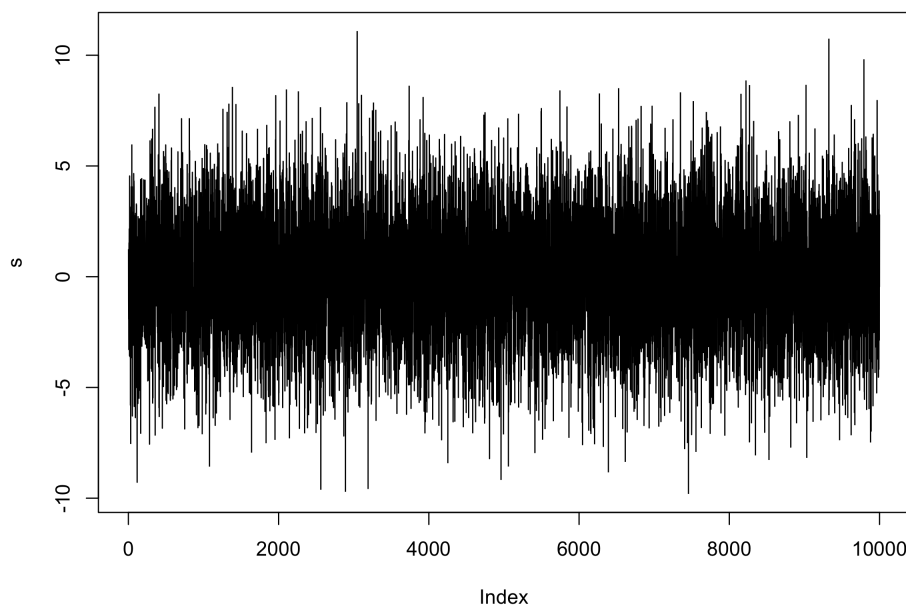


Figure 12.12: Plot of s_t , the stationary series formed via a linear combination of p_t , q_t and r_t

This should not surprise us at all as, by construction, the set of series was designed to form such a linear stationary combination. However, it is instructive to follow the tests through on simulated data as it helps us when analysing real financial data, as we will be doing so in the next section.

12.9.2 Johansen Test on Financial Data

In this section we will look at two separate sets of ETF baskets: EWA, EWC and IGE as well as SPY, IVV and VOO.

EWA, EWC and IGE

In the previous section we looked at Ernest Chan's [32](#) work on the cointegration between the two ETFs of EWA and EWC, representing baskets of equities for the Australian and Canadian economies, respectively.

He also discusses the Johansen test as a means of adding a third ETF into the mix, namely IGE, which contains a basket of natural resource stocks. The logic is that all three should in some part be affected by stochastic trends in commodities and thus may form a cointegrating relationship.

In Ernie's work he carried out the Johansen procedure using MatLab and was able to reject the hypothesis of $r \leq 2$ at the 5% level. Recall that this implies that he found evidence to support the existence of a stationary linear combination of EWA, EWC and IGE.

It would be useful to see if we can replicate the results using `ca.jo` in R. In order to do so we need to use the `quantmod` library:

```
> library("quantmod")
```

We now need to obtain backward-adjusted daily closing prices for EWA, EWC and IGE for the same time period that Ernie used:

```
> getSymbols("EWA", from="2006-04-26", to="2012-04-09")
> getSymbols("EWC", from="2006-04-26", to="2012-04-09")
> getSymbols("IGE", from="2006-04-26", to="2012-04-09")
```

We also need to create new variables to hold the backward-adjusted prices:

```
> ewaAdj = unclass(EWA$EWA.Adjusted)
> ewcAdj = unclass(EWC$EWC.Adjusted)
> igeAdj = unclass(IGE$IGE.Adjusted)
```

We can now perform the Johansen test on the three ETF daily price series and output the summary of the test:

```
> jotest=ca.jo(data.frame(ewaAdj,ewcAdj,igeAdj), type="trace",
  K=2, ecdet="none", spec="longrun")
> summary(jotest)
```

```
#####
# Johansen-Procedure #
#####
```

Test type: trace statistic , with linear trend

Eigenvalues (**lambda**):

```
[1] 0.011751436 0.008291262 0.002929484
```

Values of teststatistic **and** critical values of test:

	test	10pct	5pct	1pct
r <= 2		4.39	6.50	8.18 11.65
r <= 1		16.87	15.66	17.95 23.52
r = 0		34.57	28.71	31.52 37.22

Eigenvectors, normalised to first column:

(These are the cointegration relations)

	EWA.Adjusted.l2	EWC.Adjusted.l2	IGE.Adjusted.l2
EWA.Adjusted.l2	1.0000000	1.000000	1.000000
EWC.Adjusted.l2	-1.1245355	3.062052	3.925659
IGE.Adjusted.l2	0.2472966	-2.958254	-1.408897

Weights W:

(This **is** the loading matrix)

	EWA.Adjusted.l2	EW.C.Adjusted.l2	IGE.Adjusted.l2
EWA.Adjusted.d	-0.007134366	0.004087072	-0.0011290139
EW.C.Adjusted.d	0.020630544	0.004262285	-0.0011907498
IGE.Adjusted.d	0.026326231	0.010189541	-0.0009097034

Perhaps the first thing to notice is that the values of the trace test statistic differ from those given in the MatLab `jplv7` package used by Ernie. This is most likely because the `ca.jo` R function in the `urca` package requires our lag order to be two or greater ($K=2$), whereas in the MatLab equivalent it is possible to use a lag of unity ($K=1$).

Our trace test statistic is broadly similar for $r \leq 2$ at 4.39 versus 4.471 for Ernie's results. However the critical values are quite different, with `ca.jo` having a value of 8.18 at the 95% level for the $r \leq 2$ hypothesis, while the `jplv7` package gives 3.841. Crucially Ernie's test statistic is greater than at the 5% level, while ours is lower. Hence we do not have sufficient evidence to reject the null hypothesis of $r \leq 2$ for $K = 2$ lags.

The key issue here is that there is no difference between the two sets of time series used for each analysis! The only difference is that the implementations of the Johansen test are different between R's `urca` and MatLab's `jplv7`. This means we need to be extremely careful when evaluating the results of statistical tests, especially between differing implementations and programming languages.

SPY, IVV and VOO

Another approach is to consider a basket of ETFs that track an equity index. For instance, there are a multitude of ETFs that track the US S&P500 stock market index such as Standard & Poor's Depository Receipts SPY, the iShares IVV and Vanguard's VOO. Given that they all track the same underlying asset it is likely that these three ETFs will have a strong cointegrating relationship.

Let us obtain the daily adjusted closing prices for each of these ETFs over the last year:

```
> getSymbols("SPY", from="2015-01-01", to="2015-12-31")
> getSymbols("IVV", from="2015-01-01", to="2015-12-31")
> getSymbols("VOO", from="2015-01-01", to="2015-12-31")
```

As with EWC, EWA and IGE, we need to utilise the backward adjusted prices:

```
> spyAdj = unclass(SPY$SPY.Adjusted)
> ivvAdj = unclass(IVV$IVV.Adjusted)
> vooAdj = unclass(VOO$VOO.Adjusted)
```

Finally, let us run the Johansen test with the three ETFs and output the results:

```
> jotest=ca.jo(data.frame(spyAdj,ivvAdj,vooAdj), type="trace", K=2,
  ecdet="none", spec="longrun")
> summary(jotest)
```

```
#####
# Johansen-Procedure #
#####
```

Test type: trace statistic , with linear trend

Eigenvalues (**lambda**):

[1] 0.29311420 0.24149750 0.04308716

Values of teststatistic **and** critical values of test:

	test	10pct	5pct	1pct
$r \leq 2$		11.01	6.50	8.18 11.65
$r \leq 1$		80.11	15.66	17.95 23.52
$r = 0$		166.83	28.71	31.52 37.22

Eigenvectors, normalised to first column:

(These are the cointegration relations)

	SPY.Adjusted.l2	IVV.Adjusted.l2	V00.Adjusted.l2
SPY.Adjusted.l2	1.0000000	1.0000000	1.0000000
IVV.Adjusted.l2	-0.3669563	-4.649203	-0.5170538
V00.Adjusted.l2	-0.6783666	4.042139	-0.2426406

Weights W:

(This **is** the loading matrix)

	SPY.Adjusted.l2	IVV.Adjusted.l2	V00.Adjusted.l2
SPY.Adjusted.d	-1.8692017	0.2875776	-0.3086708
IVV.Adjusted.d	-1.2062706	0.3965064	-0.3160481
V00.Adjusted.d	-0.9414142	0.2182340	-0.2871731

As before we sequentially carry out the hypothesis tests beginning with the null hypothesis of $r = 0$ versus the alternative hypothesis of $r > 0$. There is clear evidence to reject the null hypothesis at the 1% level and we can likely conclude that $r > 0$.

Similarly when we carry out the $r \leq 1$ null hypothesis versus the $r > 1$ alternative hypothesis we have sufficient evidence to reject the null hypothesis at the 1% level and can conclude $r > 1$.

However, for the $r \leq 2$ hypothesis we can only reject the null hypothesis at the 5% level. This is weaker evidence than the previous hypotheses and, although it suggests we can reject the null at this level, we should be careful that r might equal two, rather than exceed two. What this means is that it may be possible to form a linear combination with only two assets rather than requiring all three to form a cointegrating portfolio.

In addition we should be extremely cautious of interpreting these results as I have only used one years worth of data, which is approximately 250 trading days. Such a small sample is unlikely to provide a true representation of the underlying relationships. Hence, we must always be careful in interpreting statistical tests!

12.9.3 Full Code

```

library("quantmod")
library("tseries")
library("urca")

set.seed(123)

## Simulated cointegrated series

z <- rep(0, 10000)
for (i in 2:10000) z[i] <- z[i-1] + rnorm(1)

p <- q <- r <- rep(0, 10000)

p <- 0.3*z + rnorm(10000)
q <- 0.6*z + rnorm(10000)
r <- 0.8*z + rnorm(10000)

jotest=ca.jo(data.frame(p,q,r), type="trace", K=2,
  ecdet="none", spec="longrun")
summary(jotest)

s = 1.000*p + 1.791324*q - 1.717271*r
plot(s, type="l")

adf.test(s)

## EWA, EWC and IGE

getSymbols("EWA", from="2006-04-26", to="2012-04-09")
getSymbols("EWC", from="2006-04-26", to="2012-04-09")
getSymbols("IGE", from="2006-04-26", to="2012-04-09")

ewaAdj = unclass(EWA$EWA.Adjusted)
ewcAdj = unclass(EWC$EWC.Adjusted)
igeAdj = unclass(IGE$IGE.Adjusted)

jotest=ca.jo(data.frame(ewaAdj,ewcAdj,igeAdj), type="trace",
  K=2, ecdet="none", spec="longrun")
summary(jotest)

## SPY, IVV and VOO

getSymbols("SPY", from="2015-01-01", to="2015-12-31")
getSymbols("IVV", from="2015-01-01", to="2015-12-31")
getSymbols("VOO", from="2015-01-01", to="2015-12-31")

```

```
spyAdj = unclass(SPY$SPY.Adjusted)
ivvAdj = unclass(IVV$IVV.Adjusted)
vooAdj = unclass(V00$V00.Adjusted)

jotest=ca.jo(data.frame(spyAdj,ivvAdj,vooAdj), type="trace",
  K=2, ecdet="none", spec="longrun")
summary(jotest)
```